

Practical Work no. 1 – Documentation

Python

Specification

We define a layered architecture for representing and manipulating directed graphs

The program is split into 4 layers:

- Domain layer : stores the actual graph structure
- Repository layer : loads, saves in files and generates random graphs
- Service layer : exposes graph operations
- UI layer : console interface for interacting with the graph

The core graph structure is represented by the class `GraphData`, which models a directed graph using adjacency lists and cost dictionary

`GraphData` stores:

- The number of vertices
- Inbound and outbound adjacency lists = a dictionary whose keys are the vertices and the values stored are lists of vertices for each vertex key
- A dictionary of edge costs

Each vertex has

- A list of inbound neighbours
- A list of outbound neighbours

Each edge is represented by a (start, end) pair mapped to a cost

Classes used

`GraphData(nr_vertices)`

- Constructs a graph with `nr_vertices` vertices and no edges and initializes empty inbound outbound lists for each vertex in 2 dictionaries

`nr_vertices` = returns the number of vertices in the graph

`nr_edges` = returns the number of edges (size of the cost dictionary)

`in_edges` = returns the inbound adjacency dictionary

`out_edges` = returns the outbound adjacency dictionary

`costs` = returns the dictionary mapping (start, end) → cost

`copy()` = creates a deep copy of the graph, duplicating all adjacency and cost dictionaries

GraphRepo(filename)

- Loads and writes a graph in/from a text file. Also generates random graphs

`add_dict(graph, start, end, cost)` = adds an edge to the internal dictionaries of the given graph

- start and end must be valid vertices
- the edge must not already exist

`remove_dict(graph, start, end)` = removes an existing edge from the internal dictionaries of the given graph

`_write_random(filename, vertices, edges=None)` = generates a random directed graph and writes it to a file

- if edges is not specified – a random number is chosen
- if edges > vertices² an exception is raised

`_save_modifications(graph, filename)` = saves the current graph state to a file in the same format as loading

GraphService

- Provides all graph operations used by UI

`get_vertices_number()` = returns the number of vertices

`iterate_vertices()` = returns a sorted list of all vertices

`check_edge(start, end)` = returns True if an edge exists between the given vertices

`get_in_degree(vertex)` = returns the number of inbound neighbours of a vertex

`get_out_degree(vertex)` = returns the number of outbound neighbours of a vertex

`iterate_outbound(vertex)` = returns a sorted list of outbound neighbours

`iterate_inbound(vertex)` = returns a sorted list of inbound neighbours

`retrieve_cost(start, end)` = returns the cost of the edge (start, end)

`change_cost(start, end, new_cost)` = updates the cost of an existing edge and saves the modification

`add_vertex(new_vertex)` = adds a new vertex

- `new_vertex` must be exactly the next available id

`add_edge(start, end, cost)` = adds a new edge

- both vertices must exist
- the edge must not already exist

`remove_vertex(vertex)` = removes a vertex and all inbound outbound edges

`remove_edge(start, end)` = removes an existing edge

`copy_graph()` = returns a deep copy of the graph

GraphUI

- Provides a console interface for interacting with the graph

`start()` = displays the initial menu :

1. load (existing) graph from file
2. generate random graph

`load_graph()` = asks for input and modifications files and loads the graph through the repository and initializes the service

`generate_graph()` = asks for number of vertices, optional number of edges, output and modifications files and generates a random graph

`menu()` = displays the main command menu:

1. number of vertices
2. iterate vertices
3. check edge
4. in degree
5. out degree
6. iterate inbound
7. iterate outbound
8. add vertex
9. remove vertex
10. add edge
11. remove edge
12. change cost
13. retrieve cost

14. copy graph
15. display internal representations of the graph
16. exit

Implementation

The architecture follows a layered design, separating data storage, logic and UI

GraphData uses Python dictionaries and lists to store adjacency information

GraphRepo ensures file correctness and prevents invalid graphs

GraphService ensures all operations respect graph invariants

GraphUI provides a simple console interface